```
def deserves_a_look?(book)
  amazon = Amazon.new
  amazon.reviews_of(book).size > 20
end
```

This code works in most cases, but it doesn't manage exceptions. If a remote call to Amazon fails, Bookworm itself should log this problem and proceed. You could easily add exception management to each line in Bookworm that calls deserves_a_look?—but there are tens of such lines, and you don't want to change all of them.

To sum up the problem: you have a method that you don't want to modify directly because it's in a library. You want to wrap additional functionality around this method so that all clients get the additional functionality automatically. You can do this in a few ways, but to get to the first of them you need to know about *aliases*.

Around Aliases

You can give an alternate name to a Ruby method by using Module#alias_method:

class_definitions/alias.rb

```
class MyClass
  def my_method; 'my_method()'; end
  alias_method :m, :my_method
end

obj = MyClass.new
obj.my_method   # => "my_method()"
obj.m           # => "my_method()"
```

In alias_method, the new name for the method comes first, and the original name comes second. You can provide the names either as symbols or as strings.

(Ruby also has an alias keyword, which is an alternative to Module#alias_method. It can be useful if you want to alias a method at the top level, where Module#alias_method is not available.)

Continuing with the previous example:

```
class MyClass
  alias_method :m2, :m
end

obj.m2          # => "my_method()"
```

Aliases are common everywhere in Ruby, including the core libraries. For example, String#size is an alias of String#length, and the Integer class has a method with no fewer than five different names. (Can you spot it?)